



Universidad Autónoma de Baja California  
Facultad de Ingeniería Arquitectura y Diseño

Ingeniería en Software y Tecnologías Emergentes

Patrones de Software  
//Análisis de Algoritmos  
Jonathan Crespo Ragland  
// Hector Zatarain Aceves

Danna Sofia Garcia Daniels  
Ahhmed Affif Jalife Burgueño  
Fernanda Itzel Flores Valenzuela  
Raul Alejandro Trenado Sanchez  
Paul Isaac Aragon Palafox

2026-1. Grupo 952

### Requerimientos.

El videojuego cuenta con un sistema de objetos, donde el jugador puede recolectar objetos, estos se deben poder ordenar primero por su categoría, es decir si es una espada o no estos irán juntos y después se ordenarán por orden alfabético dentro de estas categorías. Cada que el jugador recoja un objeto este se deberá de ordenar según esas reglas.

### Observaciones.

Al ser un N muy pequeño de máximo 20 objetos el algoritmo de ordenamiento como tal no afecta a la compilación del videojuego.

El juego está diseñado para linux utilizando herramientas que en windows no funcionan o son difíciles de configurar o adaptar.

### Propuesta.

Nosotros proponemos Insertion Sort para este algoritmo, en el peor caso este será de  $O(n^2)$ , máximo 400 comparaciones. También es necesario una función que compare la categoría de ambos objetos para saber si van juntos. Para saber que letra va primero que otra usaremos de la librería <string.h> la función strcmp, esta comparara la primera letra por su valor ASCII con la primera letra del otro objeto

Primero trabajaremos con constantes globales, estas nos permiten mantener el orden en el código y evitar los números mágicos.

```
4  #define MAX_ITEMS 20
5  #define MAX_NAME 50
6  #define MAX_CATEGORY 20
7
```

manejaremos los ítems como Structs claro que esta es una versión simple del ítem ya ustedes agregan o adaptan lo que hace cada ítem, en este caso guardamos su nombre y su categoría.

```
8  typedef struct
9  {
10     char name[MAX_NAME];
11     char category[MAX_CATEGORY];
12 } Item;
```

En esta función compararemos las categorías de los ítems, lo primero que hace es comparar las categorías de ambos objetos con strcmp. Si las categorías son diferentes, ya tiene su respuesta ahí mismo y termina. El único caso en el que sigue adelante es cuando las dos categorías son exactamente iguales, por ejemplo dos "Weapon". Ahí ya no puede decidir por categoría porque son la misma, entonces pasa a comparar los nombres y el que vaya primero alfabéticamente es el que gana.

```
14  int compare_items(Item a, Item b)
15  {
16      int cat_cmp = strcmp(a.category, b.category);
17      if (cat_cmp != 0)
18          return cat_cmp;
19      return strcmp(a.name, b.name);
20  }
```

Función de Insertion Sort. Esta recibe dos variables de tipo item, y un puntero a un contador.

```
22  void insertionSort(Item inventory[], int *count, Item new_item)
23  {
24      int i = *count;
25
26      while (i > 0 && compare_items(inventory[i - 1], new_item) > 0)
27      {
28          inventory[i] = inventory[i - 1];
29          i--;
30      }
31
32      inventory[i] = new_item;
33      (*count)++;
34  }
```

Aquí les dejamos la misma función de Insertion Sort pero sin el nombre de las variables que escogimos.

```
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        /* Move elements of arr[0..i-1], that are
           greater than key, to one position ahead
           of their current position */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

## *Recomendaciones de código*

- `game_rules.c`

Cómo está el código ahorita se parece mucho al `command.c`. No tiene nada funcional realmente, todo está comentado ahorita. Estaría bien si agregaran las reglas o algo si no, no es necesario tener el archivo y se puede borrar.

- `game_actions_take`

Veo que en el game action tienen una doble llave que no es necesaria.

- `game_loop.c`: `command_to_string` y `game_loop_command_to_str` hacen lo mismo

Es que son dos funciones que literal hacen casi lo mismo y se podría adaptar a una sola función, yo recomendaría borrar la de game loop y quedarse con la de `command_to_string` de `command.c`.

- `srand`

`srand` debería estar en `game_create`, no en `main`, o podría ser en el `game_loop_init`, así para que cuando el juego se reinicie la semilla también se reinicia.

## *Recomendaciones de algoritmos*

- `set.c`

En `set_create` tienen un loop que itera hasta `SIZE` y recorre 256 elementos y podrían poner que sea de `i` hasta `n_ids` para reducir iteraciones innecesarias.

por ejemplo: `for (i = 0; i < set->n_ids; i++)`

Así solo itera por los ids que existen. Igual en `set_find_id` y `set_print`.

En `set_add_value` no están verificando límites ni valores duplicados. Esto les puede causar mil problemas con límites pueden tener un buffer overflow y con valores duplicados corrupciones.

- `game.c`

En `game_remove_object` están dejando un nulo en medio del array. Los loops que iteran hasta `n_objects` van a tocar ese nulo y probablemente crashear.

O sea si tienes un arreglo [`objeto0`, `objeto1`, `objeto2`, `objeto3`] y borras el `objeto1` quedaría [`objeto0`, `NULL`, `objeto2`, `objeto3`].

```
/*comprobar que esto esté bien vvv*/  
game->object[index] = NULL;
```

game\_get\_character\_id llama a game\_get\_character\_location dentro de un loop con complejidad de  $O(n^2)$

Un tip podría ser guardar la ubicación en el Character struct, cómo player que tiene su propio location. Porque ahorita hace un loop para encontrar su ubicación.

Otra cosa, game\_next\_turn se podría simplificar con un módulo para que escale mejor si agregan jugadores dinámicos.

```
Status game_next_turn(Game *game){  
    if (!game || game->n_players <= 0) {return ERROR;}  
  
    if (game->turn >= game->n_players - 1)  
    {  
        game->turn = 0;  
    }  
    else  
    {  
        game->turn++;  
    }  
  
    return OK;  
}
```

game->turn = (game->turn + 1) % game->n\_players;

- game\_actions.c

Algo que les recomiendo demasiado es cambiar su game\_actions\_attack. Ahorita como esta esta haciendo mil cosas, pueden simplificarlo y hacerlo más mantenible pueden hacer funciones separadas como:

```
enemy = game_actions_get_enemy_at(game, player_location);
```

```
damage = game_actions_roll_damage();
```

```
game_actions_apply_damage(game, player, enemy, damage);
```

```
game_actions_check_deaths(game, player, enemy);
```

**Bibliografías.**

GeeksforGeeks. (2013, March 7). *Insertion Sort Algorithm*. GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>